

Guide Administrateur — mobiTrace

v2.0

Ce document est destiné aux administrateurs système et aux référents techniques en charge de l'exploitation de l'application mobiTrace (Traçabilité, Réemploi et Affectation Circulaire des Équipements). Il couvre l'ensemble des paquets Debian du projet, y compris le module d'impression Zebra.

1. Architecture Technique

L'application repose sur une architecture modulaire distribuée en 5 paquets Debian indépendants. Le paquet principal `trace-server` constitue le cœur du système ; les autres sont optionnels.

1.1 Couches du serveur principal (trace-server)

- **Serveur Web (Front)** : Nginx assurant le SSL, le reverse proxy, le Rate Limiting et l'authentification des routes d'impression.
- **Backend (API)** : PostgREST transformant le schéma PostgreSQL en API REST sécurisée par JWT.
- **Base de données** : PostgreSQL 17 avec gestion des rôles (RLS), Triggers d'audit et logique métier.
- **Protection** : Fail2Ban surveillant les journaux Nginx pour bannir les attaques par force brute.

1.2 Modules complémentaires

- **Module impression (trace-zebra-printer)** : Microservice Python/Flask (Gunicorn, port 5050) + CUPS pour l'impression directe ZPL sur imprimantes Zebra réseau.
- **Sauvegarde externalisée (trace-backup-server / trace-backup-client)** : Partage NFS entre le serveur TRACE et une machine distante dédiée.
- **Surveillance sauvegardes (trace-backup-server-survey)** : Sonde Cron quotidienne à 08h00 envoyant un rapport par email via Postfix.

2. Installation et Déploiement

Le déploiement est entièrement automatisé via les paquets Debian (.deb).

2.1 Installation du serveur principal

```
sudo apt update
sudo apt install ./trace-server_1.0.0_all.deb
```

Le système résout automatiquement les dépendances et lance le script `postinst`.

2.2 Actions automatisées (Post-installation)

1. **Secrets** : Génération aléatoire de `DB_PASS` et `JWT_SECRET` (stockés dans `/etc/trace-api.conf`).
2. **Base de données** : Création de `parc_mobilier` et import des référentiels CSV.

3. **SSL** : Génération de certificats auto-signés dans /etc/ssl/trace/.
4. **Sécurité** : Configuration des prisons Fail2Ban et des zones de limitation Nginx.
5. **Identifiants** : Création du compte administrateur initial (admin@gouv.fr) — affiché en fin d'installation.

2.3 Installation des modules complémentaires

Ordre recommandé pour une infrastructure complète :

Ordre	Paquet	Machine cible	Prérequis
1	trace-server	Serveur TRACE	—
2	trace-backup-server	Machine NFS distante	—
3	trace-backup-client	Serveur TRACE	trace-backup-server installé
4	trace-zebra-printer	Serveur TRACE	trace-server installé, imprimante Zebra accessible réseau
5	trace-backup-server-survey	Machine NFS distante	trace-backup-server installé

Chaque paquet pose des questions interactives via debconf (adresse IP, email, relais SMTP). Répondre précisément pour éviter une reconfiguration manuelle.

3. Stratégie de Sauvegarde et Restauration

La continuité des données est assurée par des scripts dédiés et, optionnellement, par une externalisation NFS vers une machine distante.

3.1 Sauvegarde automatisée locale (trace_backup.sh)

Le script effectue un dump compressé (-Fc) et nettoie les fichiers de plus de 30 jours.

- **Point de montage** : /mnt/savetrace
- **Planification** : Tous les jours à 02h00 et 13h00 sous l'utilisateur postgres.
- Lancement manuel

```
sudo -u postgres /usr/local/bin/trace_backup.sh
```

3.2 Restauration interactive (trace_restore.sh)

Un outil interactif permet de visualiser et de sélectionner une sauvegarde à restaurer.

```
sudo /usr/local/bin/trace_restore.sh
```

Fonctionnement :

1. Affiche la liste des fichiers .dump triés par date.
2. Demande une confirmation explicite (saisie de "OUI").
3. Termine les sessions actives sur la base de données.
4. Restaure la structure et les données.

3.3 Externalisation NFS (trace-backup-server / trace-backup-client)

Si les paquets NFS sont installés, les dumps sont écrits sur /mnt/savetrace qui est monté automatiquement (systemd automount) sur le partage NFS distant.

- Vérifier le montage

```
systemctl status mnt-savetrace.automount  
mount | grep savetrace
```

- Forcer un montage manuel

```
sudo mount /mnt/savetrace
```

3.4 Surveillance des sauvegardes (trace-backup-server-survey)

Si le paquet est installé sur la machine NFS, un rapport est envoyé par email chaque jour à 08h00.

- **Script** : /usr/local/bin/mobitrace_backup_survey.sh
- **Cron** : /etc/cron.d/mobitrace-backup-survey
- Modifier les destinataires

```
sudo nano /usr/local/bin/mobitrace_backup_survey.sh  
# Modifier la variable EMAIL_DEST
```

Trois niveaux d'alerte : [OK] si 2 sauvegardes trouvées, [AVERTISSEMENT] si 1 seule, [ALERTE CRITIQUE] si aucune.

4. Sécurité et Protection Active

4.1 Gestion des bannissements (Fail2Ban)

Le système protège la route /api/rpc/login. Si une IP échoue 5 fois en 10 minutes, elle est bannie 1 heure.

- Vérifier les IPs bannies

```
sudo fail2ban-client status mobitrace-login
```

- Débannir un utilisateur

```
sudo fail2ban-client set mobitrace-login unbanip <ADRESSE_IP>
```

4.2 Audit et Traçabilité

Chaque modification (Mobilier, Catalogue, Administration) déclenche un trigger SQL qui enregistre :

- L'auteur (Email extrait du JWT)
- L'adresse IP source (transmise par Nginx via les headers)
- Le détail des champs modifiés (Ancienne vs Nouvelle valeur)

Purge des logs : Une tâche Cron (/etc/cron.d/trace_purge_logs) supprime quotidiennement les entrées d'audit de plus de 3 mois pour optimiser les performances.

5. Module d'impression Zebra (CUPS)

Le paquet trace-zebra-printer déploie un microservice d'impression permettant d'envoyer des étiquettes ZPL directement sur une imprimante Zebra réseau depuis l'interface mobiTrace.

5.1 Architecture du module

Composant	Emplacement	Rôle
Flask/Gunicorn	/opt/mobitrace-print/print_api.py Port 127.0.0.1:5050	Reçoit les requêtes de l'interface web et les transmet à CUPS.
CUPS	Port 631 (réseau local) Imprimante : Zebra_Trace	Gère la file d'attente et envoie les données ZPL en mode RAW (port 9100).
Nginx	/etc/nginx/sites-available/trace	Protège les routes /imprimer-zpl et /imprimante/* par auth JWT (cookie trace_token).
Sudoers	/etc/sudoers.d/mobitrace-print	Autorise www-data à piloter CUPS (lpadmin, cancel) sans mot de passe.

5.2 Installation

```
sudo apt install ./trace-zebra-printer_1.0.0_all.deb
```

debconf demande l'adresse IP de l'imprimante Zebra (défaut : 192.168.1.99). Cette valeur peut être modifiée ultérieurement sans réinstaller le paquet.

5.3 Vérifier l'état du module

Service mobitrace-print (microservice Flask)

```
sudo systemctl status mobitrace-print
```

Le service doit afficher active (running). En cas d'erreur :

```
sudo journalctl -u mobitrace-print -f --no-pager
```

Service CUPS

```
sudo systemctl status cups  
lpstat -p Zebra_Trace
```

La commande lpstat doit afficher :

```
printer Zebra_Trace is idle. enabled since ...
```

5.4 Gérer l'imprimante CUPS

Interface d'administration CUPS (navigateur)

CUPS expose une interface web d'administration accessible sur le réseau local :

```
http://<IP_SERVEUR>:631
```

L'accès à l'interface d'administration CUPS (/admin) nécessite les identifiants système de la machine serveur (utilisateur Linux avec droits sudo).

Vérifier la file d'attente

```
# Liste des jobs en attente
lpq -P Zebra_Trace

# Statut détaillé
lpstat -o Zebra_Trace
```

Annuler tous les jobs en attente

```
sudo cancel -a Zebra_Trace
```

Annuler un job spécifique

```
# Récupérer l'ID du job avec lpq, puis :
sudo cancel <ID_JOB>
```

Mettre l'imprimante en pause / reprendre

```
# Pause
sudo cupsdisable Zebra_Trace

# Reprise
sudo cupsenable Zebra_Trace
```

5.5 Modifier l'adresse IP de l'imprimante

Deux méthodes sont disponibles :

Méthode 1 — Via l'API mobiTrace (recommandée)

Depuis l'interface d'administration mobiTrace, section Impression, le champ IP permet de reconfigurer l'imprimante à chaud sans redémarrage de service.

Méthode 2 — Via la ligne de commande

```
# Remplacer 192.168.1.99 par la nouvelle IP
sudo lpadmin -p Zebra_Trace -v socket://192.168.1.99:9100

# Vérifier la modification
lpstat -v Zebra_Trace
```

5.6 Ajouter ou remplacer l'imprimante CUPS

En cas de remplacement physique de l'imprimante (nouveau modèle Zebra) :

```
# Supprimer l'ancienne définition
sudo lpadmin -x Zebra_Trace

# Recréer avec la nouvelle IP
sudo lpadmin -p Zebra_Trace -E -v socket://<NOUVELLE_IP>:9100 -m raw

# Réactiver
sudo cupsenable Zebra_Trace
```

Toujours conserver le nom Zebra_Trace. Le microservice Flask référence ce nom en dur dans /opt/mobitrace-print/print_api.py (variable PRINTER). Modifier le nom nécessite de mettre à jour le fichier print_api.py et de redémarrer mobitrace-print.

5.7 Dépannage impression

Symptôme	Vérification	Solution
L'interface affiche "Imprimante injoignable"	lpstat -p Zebra_Trace	Vérifier que l'imprimante est allumée, en réseau, et que le port 9100 est accessible : nc -zv <IP> 9100
Job envoyé mais rien ne s'imprime	lpq -P Zebra_Trace journalctl -u cups	Vérifier que le mode RAW est actif (lptions -p Zebra_Trace -l). Recréer l'imprimante avec -m raw si nécessaire.
Erreur 401 sur /imprimer-zpl	Navigateur : vérifier la présence du cookie trace_token	Session expirée. Se reconnecter à mobiTrace. Vérifier la config /_zebra-auth dans Nginx.
Erreur 500 depuis l'interface	journalctl -u mobitrace-print -n 50	Redémarrer le service : sudo systemctl restart mobitrace-print
mobitrace-print ne démarre pas	journalctl -u mobitrace-print -b	Vérifier que Flask et Gunicorn sont installés : python3 -m flask --version && gunicorn --version

5.8 Désinstallation du module Zebra

```
# Suppression simple (conserve CUPS et l'imprimante)
sudo apt remove trace-zebra-printer

# Suppression complète avec nettoyage CUPS
sudo apt purge trace-zebra-printer
sudo lpadmin -x Zebra_Trace
```

6. Maintenance et Diagnostic

6.1 Gestion des services

Service	Commande de redémarrage	Usage
Nginx	systemctl restart nginx	Modification de la config SSL, Proxy ou ajout de routes impression.
PostgREST (trace-api)	systemctl restart trace-api	Prise en compte de nouveaux GRANTS SQL.
Fail2Ban	systemctl restart fail2ban	Mise à jour des règles de filtrage.
CUPS	systemctl restart cups	Modification de la configuration d'impression ou ajout d'imprimante.
mobitrace-print	systemctl restart mobitrace-print	Redémarrage du microservice Flask après modification de print_api.py.

6.2 Analyse des journaux

- **Logs d'accès web** : /var/log/nginx/access.log (codes 400/401/429)
- **Logs erreurs API** : journalctl -u trace-api -f

- **Logs sécurité** : /var/log/fail2ban.log
- **Logs impression Flask** : journalctl -u mobitrace-print -f
- **Logs CUPS** : journalctl -u cups -f ou /var/log/cups/error_log

6.3 Vérification globale rapide

```
# État de tous les services mobiTrace
systemctl status nginx trace-api fail2ban cups mobitrace-print

# Espace disque sauvegardes
df -h /mnt/savetrace
ls -lh /mnt/savetrace/
```

7. Procédure de Désinstallation

7.1 Paquet principal (trace-server)

- **Suppression simple** : sudo apt remove trace-server (conserve la base de données et les sauvegardes)
- **Purge complète** : sudo apt purge trace-server

La purge complète supprime :

1. La base de données PostgreSQL parc_mobilier
2. Les certificats SSL (/etc/ssl/trace/)
3. Les configurations Fail2Ban et Nginx
4. Le fichier de configuration PostgREST (/etc/trace-api.conf)

Les sauvegardes dans /mnt/savetrace sont conservées par sécurité dans tous les cas.

7.2 Module d'impression (trace-zebra-printer)

```
# Suppression simple
sudo apt remove trace-zebra-printer

# Purge complète (supprime aussi la configuration CUPS)
sudo apt purge trace-zebra-printer
```

7.3 Modules de sauvegarde

```
# Sur la machine NFS
sudo apt purge trace-backup-server
sudo apt purge trace-backup-server-survey

# Sur le serveur TRACE
sudo apt purge trace-backup-client
```

Avant de purger trace-backup-server, s'assurer que les dumps existants dans /var/nfs/backupTRACE ont bien

été archivés ou transférés.